

Wave-14 discovery-pressure audit — captured QA evidence (§11.4.83)

Revision: 1 **Last modified:** 2026-07-10T10:20:00Z

Run-id: wave14-20260710. Independent adversarial audits (§11.4.118 loop-until-dry) of the largest remaining un-deep-audited surfaces, each closure carrying real RED → GREEN + **conductor-run** polarity (§11.4.115) + independent gates (§11.4.125/§11.4.142/§11.4.134). Subagents ran zero git; the conductor is the independent review + polarity + commit seam (§11.4.20/§11.4.70). Both fixes below came from GENUINELY-FRESH coverage (packages never previously 2nd-pass audited) — the loop is still yielding real defects.

1. containers pkg/lazyservice — restart no-op / fabricated success (commit 9548c36, submodule)

Defect: RegisterService builds a ONE-SHOT lifecycle.LazyBooter (runs its startFn exactly once, caching first-run success); StartService drives it via booter.EnsureStarted(). StopService called compose Down and cleared lo.started[name] but NEVER reset the booter. So a normal, in-contract Start → Stop → Start hit the booter's cached-success fast path on the second Start: startServiceInternal was never invoked, compose Up was never re-issued, the service stayed genuinely DOWN — yet StartService returned nil (a fabricated success, §11.4 lifecycle-layer PASS-bluff). Secondary: GetServiceStatus kept reporting Started=true after a stop because it reads booter.Started(), which latches true forever.

Fix: after a successful Down + lo.started[name]=false, recreate the booter under the already-held lo.mu write lock: lo.booters[name] = lifecycle.NewLazyBooter(func() error { return

`lo.startServiceInternal(svc) }`) — so the next Start re-boots from a clean not-started booter. Under the same lock as every other booter mutation $\Rightarrow$ no race. Happy path unchanged.

Files: `submodules/containers/pkg/lazyservice/orchestrator.go` + new `orchestrator_restart_test.go`.

Regression guard:

`TestLazyOrchestrator_StartService_AfterStop_ActuallyRestarts` — Register $\rightarrow$ Start (Up count==1) $\rightarrow$ Stop (Down count==1) $\rightarrow$ Start again (Up count==2 AND `GetServiceStatus().Started==true`). Outcome/counter test. Pre-fix RED: “Up calls = 1, want 2” with `StartService` still returning nil.

Independent gates (conductor): `go build/vet/gofmt clean; go test -race green` (cache 1.0s / `lazyservice` 6.0s / `lifecycle` 3.4s).

Conductor polarity (§11.4.115): delete the booter-reset block (compile-safe — `lifecycle` stays imported via `RegisterService` + the booters map field, `svc` stays used via `ComposeFile/Profile/StopTimeout`) $\Rightarrow$ the restart test FAILs identically (“Up calls = 1, want 2”); build compiled; `orchestrator.go` restored byte-identical (sha256); restored tree GREEN under `-race`. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_lazyservice_restart.py`.

Published: 9548c36 FF-pushed github+gitlab (f25228f..9548c36). Parent gitlink bump in the Wave-14 consolidation commit.

2. docs_chain internal/hash — FingerprintMembers separator-injection collision (commit 9510e01, submodule)

Defect: `FingerprintMembers` is the §11.4.86 drift-proof roster/corpus fingerprint the whole change-detection gate relies on. It computed `sha256(strings.Join(sortedMembers, "\n"))`. A bare `"\n"`-join is NOT injective — newline is a legal path byte on the POSIX targets `docs_chain` runs on, so a member containing the separator shifts a record boundary and distinct member SETS collide: `{"a", "b\nc"}` and `{"a\nb", "c"}` both encode to `"a\nb\nc"` $\rightarrow$ identical fingerprint, and `{"a\nb\nc"}` (1 member) collides with `{"a", "b", "c"}` (3 members). A code comment even asserted the separator “cannot appear inside a member path” — factually false. Impact: a §11.4.86 roster/corpus sidecar that transitions across a newline-boundary-preserving

reshuffle is reported UNCHANGED → the Status/Summary doc + HTML/PDF exports are NOT resynced → a STALE export ships past the drift gate (a §11.4 PASS-bluff at the fingerprint primitive). Exported reusable primitive (§11.4.28) whose contract must hold for ALL inputs.

Fix: feed each sorted member to the digest as a fixed-width 8-byte big-endian length + its raw bytes (injective length-prefix framing — the standard defence against concatenation-boundary ambiguity). Order-independence, membership-sensitivity, determinism, non-mutation all preserved; collision class closed for every input. Blast radius (§11.4.92): the fingerprint VALUE changes for all member sets, so any existing gitignored state.json sidecar shows one-time drift and resyncs on the next run (correct gate behaviour, not breakage); no committed test pins a literal fingerprint (verified: `go test -race ./... green` across all 8 packages).

Files: docs_chain/internal/hash/hash.go + hash_test.go.

Regression guard: TestFingerprintMembers_SeparatorInjection — asserts `FingerprintMembers({"a", "b\nc"}) != FingerprintMembers({"a\nb", "c"}) AND FingerprintMembers({"a\nb\nc"}) != FingerprintMembers({"a", "b", "c"})` (cardinality). Pre-fix RED: distinct member sets share a fingerprint.

Independent gates (conductor): `go build/vet/gofmt clean; go test -race ./... green` (all 8 packages).

Conductor polarity (§11.4.115): drop the length frame and write `m + "\n"` per member instead (compile-safe — the `binary.BigEndian.PutUint64` line stays so `binary+lenbuf` remain referenced) ⇒ reproduces the newline collision ⇒ the test FAILS (distinct sets share a fingerprint); build compiled; hash.go restored byte-identical (sha256); restored tree GREEN. ALL_POLARITY_OK=True. Harness: scratchpad/polarity_docschain_hash.py.

Published: 9510e01 FF-pushed github+gitlab (394270e..9510e01). Parent gitlink bump in the Wave-14 consolidation commit.

3. Convergence — independent NO-DEFECT passes (§11.4.118 loop-until-dry)

- **server/internal/deviceemu** — NO-DEFECT. Production surface is `emulator.go` (614 LOC); 97.7% stmt coverage, `-race -count=2` deterministic. Every one of the 3 `http.Client.Do` call sites closes its body (`defer resp.Body.Close()` + full drain, incl. the 204 branch); the only ticker (`RunLoop`) is `defer t.Stop()`ed and created after the early-return; no goroutine spawned in production; all mutable Device fields (`operatorToken/deviceID/deviceToken/current/healthy`) read+written only under `d.mu`; ignored `io.ReadAll` errors cannot mask a failure (a truncated body fails the subsequent status/unmarshal check); schema-rejected telemetry surfaced in `Outcome.Note`, not hidden. No silent-error-swallow, no nil-deref, no type assertions.
- **containers pkg/cache** — CLEAN. `Manifest/ImageEntry` immutable post-load; `FilesystemStore` shared map only under `s.mu`; per-imageID mutex + `syscall.Flock` serialize in-process + cross-process; blob write is temp-file + SHA/size verify + atomic rename. (Unbounded keymus growth noted as a memory-not-correctness observation, not a defect.)
- **containers pkg/lifecycle** — CLEAN (new findings). The idle-shutdown stale-fire race is the already-fixed 1a526ab (not re-flagged); `serviceEntry` state all under `m.mu`; `LazyBooter` uses `atomic.Pointer/atomic.Int32/once`; semaphore channel-bounded. Double-boot TOCTOU + `idleCtrl` race already fixed + regression-tested.
- **docs_chain internal/{graph,orchestrator,runner}** — CLEAN. `graph.TopoOrder` is Kahn with lowest-ID tie-break + sorted adjacency (deterministic ×3 runs); cycle detection via residual in-degree → `*CycleError`; `Recompute` early-cutoff prunes when recomputed hash == baseline + forward-feeds fresh content between levels; `ResolveSync` both-dirty → `*ConflictError` (no writes); `orchestrator.Run` stage-then-atomic-commit with rollback on conflict/transform error; runner propagates transform exit+stderr, `Verify` rebinds to temp (no live mutation). No goroutines/channels/shared-mutable-state in the 4 in-scope packages → data-race class N/A.

Convergence status: Wave-14 netted **2 real fixes + 6 NO-DEFECT passes** (deviceemu, cache, lifecycle, graph, orchestrator, runner). Both fixes were §11.4-class (fabricated-success restart; fingerprint-collision drift MISS) found in genuinely-fresh coverage, so the loop is NOT yet

dry. The containers module remains the largest un-deep-audited surface (~30 packages — remote, distribution, orchestrator, monitor, health, runtime, vm, etc.); a further round targeting its highest-risk concurrency/state-heavy packages is warranted. Release tag stays gated on §11.4.185 QA-team manual confirmation + the 3 operator-blocked mirror-fork publishes.